

PREPARATION REPORT LAB2

ADVANCED CPU ARCHITECTURE AND HARDWARE

ACCELERATORS LAB

361.1.4693

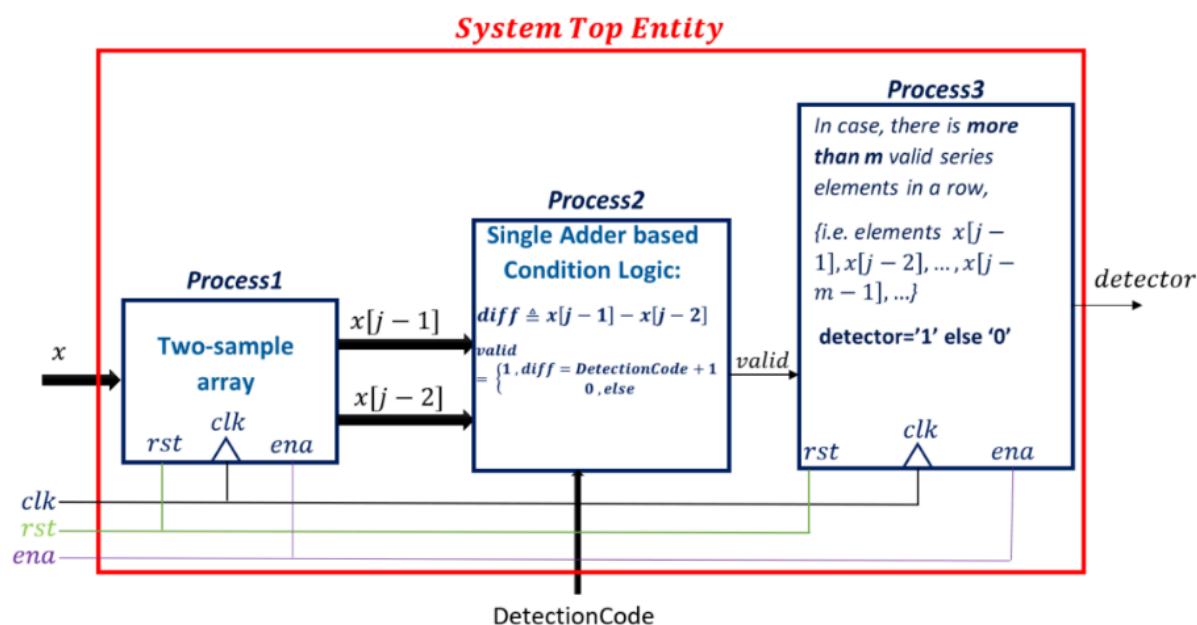
Yoav Tsur 211903406

Elad Lavi 211626551

2026 Semester B

מבוא

במעבדה זו תכננו ומימשנו מערכת דיגיטלית סינכרונית בשפת תיאור חומרה VHDL לגילוי רצפים (Sequence Detector). המערכת נועדה לזהות תתי-סדרות בתוך זרם נתונים של דגימות כאשר הקריטריון לזיהוי הוא הפרש חשבוני קבוע בין דגימות עוקבות. מימוש המערכת מתבסס על שילוב בין תיאור מבני (Structural) לתיאור התנהגותי (Behavioral), תוך שימוש ברכיב Adder גנרי לביצוע חישוב ההפרשים בשיטת המשלים ל-2. המערכת דוגמת את כניסת הנתונים באמצעות שרשרת פליפ-פלופים המסונכרנת ע"י שעון CLK. המערכת מפרידה בין שלב חישוב ההפרש לבין תהליך הספירה אשר מבוצע על ידי מונה פנימי המתקדם רק כאשר קיים התנאי של Enable פעיל וההפרש בין דגימות עוקבות תואם לקוד הגילוי שנבחר. תכנון זה מבטיח כי אות הגילוי (detector) יעלה ל-'1' לוגי רק לאחר זיהוי רצף תקין. בנוסף המערכת כוללת אות איפוס (Reset) המבטיח חזרה למצב התחלתי תקין. התכנון בוצע תוך שימוש בגנריות על מנת להתאים לכל רוחב של דגימות הנתונים ולכל רצף שנבחר וזאת ללא צורך בשינוי הלוגיקה המרכזית של המערכת. להלן דיאגרמה המייצגת את מימוש המערכת:



בנוסף מצורפת טבלה המגדירה את ההפרשים המתאימים בין כל זוג דגימות בהתאם לDetectionCode הנכנס למערכת:

Detection Code	Condition
0	$x[j-1] - x[j-2] = 1$
1	$x[j-1] - x[j-2] = 2$
2	$x[j-1] - x[j-2] = 3$
3	$x[j-1] - x[j-2] = 4$

מודול TOP

מודול זה הינו המודול אשר עוטר את המערכת כולה. הוא מקבל וקטור כניסה X ברוחב גנרי של N ביטים המייצג את זרימת הנתונים שעליהם נרצה לבצע את פעולת המערכת, מספר integer המוגדר להיות בין 0 לבין 3 שנקרא Detecion Code הנועד לקבוע את ההפרש הרצוי בין דגימות עוקבות (נשים לב כי לפי הטבלה הוא שווה תמיד להפרש בין שתי הדגימות הקודמות פחות 1) ובנוסף מקבל שלושה קווי בקרה: Clock, Reset, Enable. המערכת מוציאה כפלט קו Detector שמסמן האם אנחנו נמצאים במצב שבו יש לנו רצף תקין וחוקי של הפרשים עוקבים בין דגימות. בהתאם לדרישות המשימה, המודול נכתב באמצעות שלושה תהליכים (Processes) נפרדים המאפשרים מימוש מקבילי ויעיל בחומרה:

להלן תיאור קצר על החלוקה לתהליכים:

1. תהליך הדגימה: אחראי על דגימת זרם נתוני הקלט x . התהליך משתמש ב FlipFlop 2 המחוברים בטור כדי לאחסן את שתי הדגימות האחרונות מהעבר. תהליך זה הינו סינכרוני לשעון ומתבצע רק כאשר קו הבקרה ena נמצא ב'1' לוגי.
 2. תהליך לוגיקה וחישוב: שלב זה מבצע את הפעולה האריתמטית של חישוב ההפרש באמצעות Ripple Carry Adder בשיטת המשלים ל-2. לאחר מכן הוא משווה את התוצאה לערך הצפוי (Detecion Code). במידה וקיימת התאמה, מופעל Valid Signal. השימוש ברכיב Adder גנרי בודד עבור כל פעולות החישוב במערכת מהווה פתרון חסכוני במיוחד.
 3. תהליך ספירה: תהליך זה מנהל מונה סינכרוני. המונה מתקדם רק כאשר האות המאפשר (Enable) הוא '1' ואות התקפות (Valid) פעיל. אם אות Valid נופל או שמתבצע איפוס (Reset), המונה מתאפס מיידית ל-0. ברגע שהמונה מגיע לסף המוגדר מוצא הגלאי (Detector) משתנה ל-1.
- הפרדת המערכת לשלושה תהליכים נפרדים מאפשרת מימוש מקבילי בחומרה, כך שפעולות הדגימה, החישוב והספירה מתבצעות בו-זמנית בכל מחזור שעון. בחירה זו משפרת את יעילות המערכת ומבטיחה תגובה מהירה בזמן אמת לשינויים בזרם הנתונים.

התהליך הראשון -תהליך הדגימה:

```
sampling:process(clk, rst)
begin
  --if reset is 1 the vectors are zero
  if(rst = '1') then
    ff1 <= (others => '0');
    ff2 <= (others => '0');
  -- if there is rising edge in the clock and enable is 1 :sample x and use 2 FlipFlop to memory
  elsif (clk' event and clk = '1' ) then
    if (ena = '1') then
      -- signal only change at the end of the process.
      ff1 <= x;
      ff2 <= ff1;
    End if;
  End if;
End process;
```

תחילה נציין כי זהו תהליך סינכרוני אשר הגדרנו לו רשימת רגישויות שכוללת Clock, Reset. ניתן לשים לב שהגדרנו 2 סיגנלים ff1,ff2 שהם למעשה שימשו לנו כרכיבי הזכרון, ה FlipFlop של המערכת.

במידה והקו בקרה Reset עולה ל-1 אנחנו מאפסים את רכיבי הזכרון. דאגנו שלרכיבי הזכרון תבוצע השמה רק כאשר יש עליית שעון ובנוסף קו הבקרה Enable שווה ל-1.

התהליך השני - תהליך חישוב ההפרש והשוואה לקוד הגילוי:

```
-- (x[j-1] - x[j-2]) by cin =1 and not b as vector we use compliment 2 method.
b_not <= not(ff2);
RippleAdder : Adder generic map (length => n) port map ( a => ff1 , b => b_not , cin => '1' , s => res, cout => Cout ) ;

--Convert and compare (detectionCode +1) to n bit vector becasue the table ,check validation by condition.
valid <= '1' when (res = conv_std_logic_vector(DetectionCode + 1 ,n)) else '0';
```

בשונה מתהליכי הדגימה והספירה הסינכרוניים, תהליך זה ממומש בלוגיקה קומבינטורית ללא שימוש במילה השמורה process, מה שמאפשר חישוב מיידי של הנתונים. למעשה, בתהליך זה אנחנו מבצעים בדיקה אם ההפרש בין שתי הדגימות השמורות לנו מקיים את התנאי הרלוונטי ל Detection Code שנקלט במערכת, כפי שמצויין בתמונה למעלה של הטבלה. לצורך בדיקת ההפרש השתמשנו במודל ADDER שניתן לנו מראש. כדי לבצע חיבור דאגנו לבצע פעולה לוגית של שלילה על הוקטור b אשר הוא המספר המחסר ולהכניס Cin =1 כנשא. אחכ הגדרנו סיגנל עזר שנקרא Valid אותנו הגדרנו להיות 1 כאשר תוצאת החיסור תואמת בדיוק להפרש הנדרש.

התהליך השלישי - ניהול המונה והפקת אות הגילוי

```
valid_series : process(clk , rst )
begin
    ---if there is reset to the clock reset counter --> detector =0.
    if(rst = '1') then
        Counter <= 0;
    --- if there is rising edge and enable "high" we handle the counter.
    elsif (clk' event and clk = '1' ) then
        if (ena = '1') then
            if (valid = '1') then
                if (counter < m ) then
                    counter <= Counter +1;
                End if;
            else
                ----if no valid detection code reset the counter.
                counter <= 0;
            End if;
        End if;
    end if;
End process;

---change the detector by mux, outside the process for real time changes.
with Counter select
detector <= '1' when m,
            '0' when others;
```

תהליך זה למעשה פוצל לשני חלקים משלימים על מנת להבטיח ביצועים אופטימליים ותגובה מהירה של המערכת:

1. עדכון המונה: חלק זה אחראי על ניהול המונה הפנימי בסנכרון לשעון המערכת ולקו האיפוס האסינכרוני (Reset). המונה סופר את רצף הביטים התקינים המגיעים מהסיגנל Valid. פעולת הספירה מותנית בסיגנל Enable פעיל; כאשר הוא נמוך, המונה "קופא" ושומר על ערכו. במידה וסיגנל ה-Valid יורד ל-'0' (שבירת הרצף) או שמתקבל אות איפוס, המונה חוזר מיידית ל-0.

2. הפקת אות הגילוי: עדכון אות המוצא detector מתבצע מחוץ לתהליך הסינכרוני באמצעות פקודת with-select קומבינטורית. בחירה זו נובעת מהרצון למנוע השהיה (Delay) של מחזור שעון נוסף. אם היינו מעדכנים את ה-detector בתוך ה-process, הערך היה משתנה רק בעליית השעון הבאה לאחר הגעת המונה לערך המטרה. הוצאת התנאי אל מחוץ לתהליך מבטיחה תגובה בזמן אמת.

תוצאות הסימולציה:

במשימה קיבלנו שלושה קבצי TB שנכתבו מראש אותם התבקשנו להריץ על הTOP אותו אנו כתבנו ולהשוות את קבצי הLIST בתוכנה TextDiff להלן התוצאות של שלושת ההרצות: ההרצה נבדקה עבור סימולציה לאורך של 4 מיקרו שניות. תוצאות הרצת TB1:

C:\Users\yaron\Desktop\simlab2\76x208\part 1\1					C:\Users\yaron\Desktop\simlab2\76x208\part 1\1				
1	ps	/tb/rest	/tb/x		1	ps	/tb/rest	/tb/x	
2	delta	/tb/ena	/tb/detector		2	delta	/tb/ena	/tb/detector	
3		/tb/clx			3		/tb/clx		
4		/tb/DetectionCode			4		/tb/DetectionCode		
5	0 +1	1 0 0 00000000 0 0			5	0 +1	1 0 0 00000000 0 0		
6	50000 +1	1 0 1 00000000 0 0			6	50000 +1	1 0 1 00000000 0 0		
7	100000 +1	0 0 0 00000001 0 0			7	100000 +1	0 0 0 00000001 0 0		
8	150000 +1	0 0 1 00000001 0 0			8	150000 +1	0 0 1 00000001 0 0		
9	200000 +1	0 1 0 00000010 0 0			9	200000 +1	0 1 0 00000010 0 0		
10	250000 +1	0 1 1 00000010 0 0			10	250000 +1	0 1 1 00000010 0 0		
11	300000 +1	0 1 0 00000011 0 0			11	300000 +1	0 1 0 00000011 0 0		
12	350000 +1	0 1 1 00000011 0 0			12	350000 +1	0 1 1 00000011 0 0		
13	400000 +1	0 1 0 00000100 0 0			13	400000 +1	0 1 0 00000100 0 0		
14	450000 +1	0 1 1 00000100 0 0			14	450000 +1	0 1 1 00000100 0 0		
15	500000 +1	0 1 0 00000101 0 0			15	500000 +1	0 1 0 00000101 0 0		
16	550000 +1	0 1 1 00000101 0 0			16	550000 +1	0 1 1 00000101 0 0		
17	600000 +1	0 1 0 00000110 0 0			17	600000 +1	0 1 0 00000110 0 0		
18	650000 +1	0 1 0 00000110 0 0			18	650000 +1	0 1 0 00000110 0 0		
19	700000 +1	0 1 0 00000111 0 0			19	700000 +1	0 1 0 00000111 0 0		
20	750000 +1	0 1 0 00000111 0 0			20	750000 +1	0 1 0 00000111 0 0		
21	800000 +1	0 1 0 00001000 0 0			21	800000 +1	0 1 0 00001000 0 0		
22	850000 +1	0 1 0 00001000 0 0			22	850000 +1	0 1 0 00001000 0 0		
23	900000 +1	0 1 0 00001001 0 0			23	900000 +1	0 1 0 00001001 0 0		
24	950000 +1	0 1 1 00001001 0 0			24	950000 +1	0 1 1 00001001 0 0		
25	1000000 +1	0 1 0 00001010 0 0			25	1000000 +1	0 1 0 00001010 0 0		
26	1050000 +3	0 1 1 00001010 0 1			26	1050000 +3	0 1 1 00001010 0 1		
27	1100000 +1	0 1 0 00001011 0 1			27	1100000 +1	0 1 0 00001011 0 1		
28	1150000 +1	0 1 1 00001011 0 1			28	1150000 +1	0 1 1 00001011 0 1		
29	1200000 +1	0 1 0 00000000 1 0			29	1200000 +1	0 1 0 00000000 1 0		
30	1250000 +3	0 1 1 00000000 1 0			30	1250000 +3	0 1 1 00000000 1 0		
31	1300000 +1	0 1 0 00000010 1 0			31	1300000 +1	0 1 0 00000010 1 0		
32	1350000 +1	0 1 1 00000010 1 0			32	1350000 +1	0 1 1 00000010 1 0		
33	1400000 +1	0 1 0 00000100 1 0			33	1400000 +1	0 1 0 00000100 1 0		
34	1450000 +1	0 1 1 00000100 1 0			34	1450000 +1	0 1 1 00000100 1 0		
35	1500000 +1	0 1 0 00000110 1 0			35	1500000 +1	0 1 0 00000110 1 0		
36	1550000 +1	0 1 0 00000100 1 0			36	1550000 +1	0 1 0 00000100 1 0		
37	1600000 +1	0 1 0 00001000 1 0			37	1600000 +1	0 1 0 00001000 1 0		
38	1650000 +1	0 1 1 00000100 1 0			38	1650000 +1	0 1 1 00000100 1 0		
39	1700000 +1	0 1 0 00001010 1 0			39	1700000 +1	0 1 0 00001010 1 0		
40	1750000 +1	0 1 1 00000101 1 0			40	1750000 +1	0 1 1 00000101 1 0		
41	1800000 +1	0 1 0 00001100 1 0			41	1800000 +1	0 1 0 00001100 1 0		
42	1850000 +1	0 1 1 00001100 1 0			42	1850000 +1	0 1 1 00001100 1 0		
43	1900000 +1	0 1 0 00001110 1 0			43	1900000 +1	0 1 0 00001110 1 0		

נציין כי אמנם לעומת שאר הבדיקות לא מופיע רקע ירוק המסמן שאין טעויות אך מבדיקה קפדנית אין הבדלים בין קבצי הlist והתוצאות אכן זהות.

תוצאות הרצת TB2:

C:\Users\yaron\Desktop\simlab2\76x208\part 1\1					C:\Users\yaron\Desktop\simlab2\76x208\part 1\1				
1	ps	/tb2/rest	/tb2/x		1	ps	/tb2/rest	/tb2/x	
2	delta	/tb2/ena	/tb2/detector		2	delta	/tb2/ena	/tb2/detector	
3		/tb2/clx			3		/tb2/clx		
4		/tb2/DetectionCode			4		/tb2/DetectionCode		
5	0 +3	1 0 0 0000000 0 0			5	0 +3	1 0 0 0000000 0 0		
6	50000 +1	1 0 1 00000000 0 0			6	50000 +1	1 0 1 00000000 0 0		
7	100000 +1	0 0 0 00000001 0 0			7	100000 +1	0 0 0 00000001 0 0		
8	150000 +1	0 0 1 00000001 0 0			8	150000 +1	0 0 1 00000001 0 0		
9	200000 +1	0 1 0 00000010 0 0			9	200000 +1	0 1 0 00000010 0 0		
10	250000 +3	0 1 1 00000010 0 0			10	250000 +3	0 1 1 00000010 0 0		
11	300000 +1	0 1 0 00000011 0 0			11	300000 +1	0 1 0 00000011 0 0		
12	350000 +3	0 1 1 00000011 0 0			12	350000 +3	0 1 1 00000011 0 0		
13	400000 +1	0 1 0 00000100 0 0			13	400000 +1	0 1 0 00000100 0 0		
14	450000 +1	0 1 1 00000100 0 0			14	450000 +1	0 1 1 00000100 0 0		
15	500000 +1	0 1 0 00000101 0 0			15	500000 +1	0 1 0 00000101 0 0		
16	550000 +3	0 1 1 00000101 0 0			16	550000 +3	0 1 1 00000101 0 0		
17	600000 +1	0 1 0 00000110 0 0			17	600000 +1	0 1 0 00000110 0 0		
18	650000 +1	0 1 1 00000110 0 0			18	650000 +1	0 1 1 00000110 0 0		
19	700000 +1	0 1 0 00000111 0 0			19	700000 +1	0 1 0 00000111 0 0		
20	750000 +3	0 1 1 00000111 0 0			20	750000 +3	0 1 1 00000111 0 0		
21	800000 +1	0 1 0 00001000 0 0			21	800000 +1	0 1 0 00001000 0 0		
22	850000 +3	0 1 1 00001000 0 0			22	850000 +3	0 1 1 00001000 0 0		
23	900000 +1	0 1 0 00001001 0 0			23	900000 +1	0 1 0 00001001 0 0		
24	950000 +3	0 1 1 00001001 0 0			24	950000 +3	0 1 1 00001001 0 0		
25	1000000 +1	0 1 0 00001010 0 0			25	1000000 +1	0 1 0 00001010 0 0		
26	1050000 +3	0 1 1 00001010 0 1			26	1050000 +3	0 1 1 00001010 0 1		
27	1100000 +1	0 1 0 00001011 0 1			27	1100000 +1	0 1 0 00001011 0 1		
28	1150000 +1	0 1 1 00001011 0 1			28	1150000 +1	0 1 1 00001011 0 1		
29	1200000 +1	0 1 0 00000000 1 0			29	1200000 +1	0 1 0 00000000 1 0		
30	1250000 +3	0 1 1 00000000 1 0			30	1250000 +3	0 1 1 00000000 1 0		
31	1300000 +1	0 1 0 00000010 1 0			31	1300000 +1	0 1 0 00000010 1 0		
32	1350000 +3	0 1 1 00000010 1 0			32	1350000 +3	0 1 1 00000010 1 0		
33	1400000 +1	0 1 0 00000100 1 0			33	1400000 +1	0 1 0 00000100 1 0		
34	1450000 +3	0 1 1 00000100 1 0			34	1450000 +3	0 1 1 00000100 1 0		
35	1500000 +1	0 1 0 00000110 1 0			35	1500000 +1	0 1 0 00000110 1 0		
36	1550000 +3	0 1 1 00000110 1 0			36	1550000 +3	0 1 1 00000110 1 0		
37	1600000 +1	0 1 0 00001000 1 0			37	1600000 +1	0 1 0 00001000 1 0		
38	1650000 +3	0 1 1 00001000 1 0			38	1650000 +3	0 1 1 00001000 1 0		
39	1700000 +1	0 1 0 00001010 1 0			39	1700000 +1	0 1 0 00001010 1 0		
40	1750000 +3	0 1 1 00001010 1 0			40	1750000 +3	0 1 1 00001010 1 0		
41	1800000 +1	0 1 0 00001100 1 0			41	1800000 +1	0 1 0 00001100 1 0		
42	1850000 +3	0 1 1 00001100 1 0			42	1850000 +3	0 1 1 00001100 1 0		
43	1900000 +1	0 1 0 00001110 1 0			43	1900000 +1	0 1 0 00001110 1 0		
44	1950000 +3	0 1 1 00001110 1 0			44	1950000 +3	0 1 1 00001110 1 0		

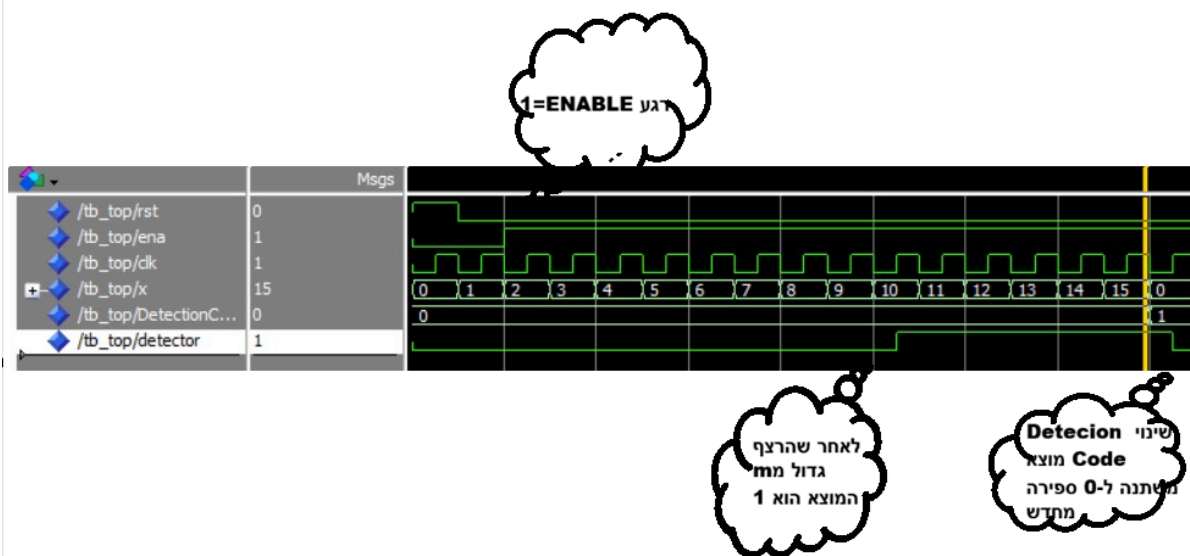
תוצאות הרצת TB3:

C:\Users\yaron\Desktop\simlab2\76x208\part 1\1					C:\Users\yaron\Desktop\simlab2\76x208\part 1\1				
1	ps	/tb3/rest	/tb3/x		1	ps	/tb3/rest	/tb3/x	
2	delta	/tb3/ena	/tb3/detector		2	delta	/tb3/ena	/tb3/detector	
3		/tb3/clx			3		/tb3/clx		
4		/tb3/DetectionCode			4		/tb3/DetectionCode		
5	0 +1	1 0 0 00000000 0 0			5	0 +1	1 0 0 00000000 0 0		
6	50000 +1	1 0 1 00000000 0 0			6	50000 +1	1 0 1 00000000 0 0		
7	100000 +1	0 0 0 00000001 0 0			7	100000 +1	0 0 0 00000001 0 0		
8	150000 +1	0 0 1 00000001 0 0			8	150000 +1	0 0 1 00000001 0 0		
9	200000 +1	0 1 0 00000010 0 0			9	200000 +1	0 1 0 00000010 0 0		
10	250000 +1	0 1 1 00000010 0 0			10	250000 +1	0 1 1 00000010 0 0		
11	300000 +1	0 1 0 00000011 0 0			11	300000 +1	0 1 0 00000011 0 0		
12	350000 +1	0 1 1 00000011 0 0			12	350000 +1	0 1 1 00000011 0 0		
13	400000 +1	0 1 0 00000100 0 0			13	400000 +1	0 1 0 00000100 0 0		
14	450000 +1	0 1 1 00000100 0 0			14	450000 +1	0 1 1 00000100 0 0		
15	500000 +1	0 1 0 00000101 0 0			15	500000 +1	0 1 0 00000101 0 0		
16	550000 +1	0 1 1 00000101 0 0			16	550000 +1	0 1 1 00000101 0 0		
17	600000 +1	0 1 0 00000110 0 0			17	600000 +1	0 1 0 00000110 0 0		
18	650000 +1	0 1 1 00000110 0 0			18	650000 +1	0 1 1 00000110 0 0		
19	700000 +1	0 1 0 00000111 0 0			19	700000 +1	0 1 0 00000111 0 0		
20	750000 +1	0 1 1 00000111 0 0			20	750000 +1	0 1 1 00000111 0 0		
21	800000 +1	0 1 0 00001000 0 0			21	800000 +1	0 1 0 00001000 0 0		
22	850000 +1	0 1 1 00001000 0 0			22	850000 +1	0 1 1 00001000 0 0		
23	900000 +1	0 1 0 00001001 0 0			23	900000 +1	0 1 0 00001001 0 0		
24	950000 +1	0 1 1 00001001 0 0			24	950000 +1	0 1 1 00001001 0 0		
25	1000000 +1	0 1 0 00001010 0 0			25	1000000 +1	0 1 0 00001010 0 0		
26	1050000 +3	0 1 1 00001010 0 1			26	1050000 +3	0 1 1 00001010 0 1		
27	1100000 +1	0 1 0 00001011 0 1			27	1100000 +1	0 1 0 00001011 0 1		
28	1150000 +1	0 1 1 00001011 0 1			28	1150000 +1	0 1 1 00001011 0 1		
29	1200000 +1	0 1 0 00000000 0 1			29	1200000 +1	0 1 0 00000000 0 1		
30	1250000 +1	0 1 1 00000000 0 1			30	1250000 +1	0 1 1 00000000 0 1		
31	1300000 +1	0 1 0 00000010 0 1			31	1300000 +1	0 1 0 00000010 0 1		
32	1350000 +1	0 1 1 00000010 0 1			32	1350000 +1	0 1 1 00000010 0 1		
33	1400000 +1	0 1 0 00000010 0 0			33	1400000 +1	0 1 0 00000010 0 0		
34	1450000 +1	0 1 1 00000010 0 0			34	1450000 +1	0 1 1 00000010 0 0		
35	1500000 +1	0 1 0 00000011 0 0			35	1500000 +1	0 1 0 00000011 0 0		
36	1550000 +1	0 1 1 00000011 0 0			36	1550000 +1	0 1 1 00000011 0 0		
37	1600000 +1	0 1 0 00000100 0 0			37	1600000 +1	0 1 0 00000100 0 0		
38	1650000 +1	0 1 1 00000100 0 0			38	1650000 +1	0 1 1 00000100 0 0		
39	1700000 +1	0 1 0 00000101 0 0			39	1700000 +1	0 1 0 00000101 0 0		
40	1750000 +1	0 1 1 00000101 0 0			40	1750000 +1	0 1 1 00000101 0 0		
41	1800000 +1	0 1 0 00000100 0 0			41	1800000 +1	0 1 0 00000100 0 0		
42	1850000 +1	0 1 1 00000100 0 0			42	1850000 +1	0 1 1 00000100 0 0		
43	1900000 +1	0 1 0 00000101 0 0			43	1900000 +1	0 1 0 00000101 0 0		
44	1950000 +1	0 1 1 00000101 0 0			44	1950000 +1	0 1 1 00000101 0 0		

TB_top

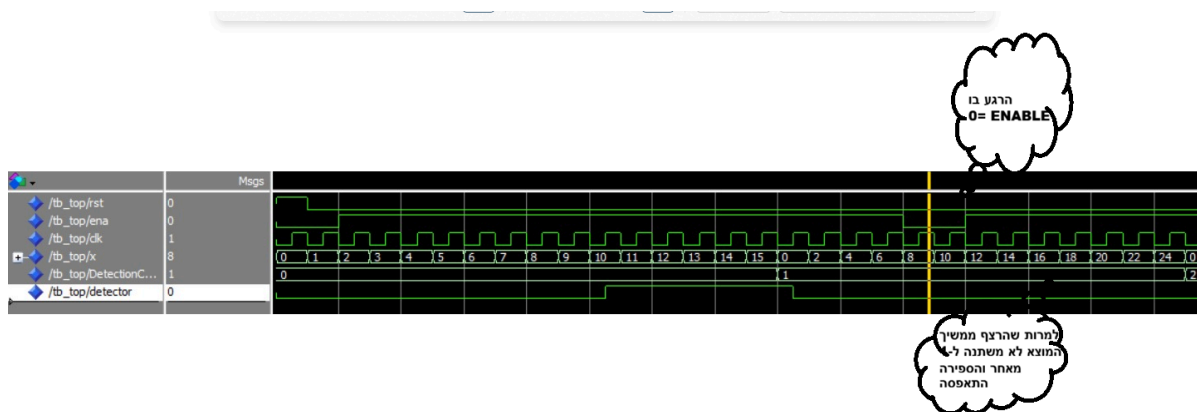
בנוסף התבקשנו לכתוב TB משלנו שיבדוק את הTOP אותו כתבנו.
הכללנו מקרה קצה ומיזגנו מקרים אפשריים כדי לבדוק אם מוצא המערכת אכן מתקבל כפי שצריך.
למעשה בדקנו 4 מקרים שונים :

1. המקרה הראשון הוא למעשה מצב בו הכל תקין, בדיקה שאכן מוצא המערכת נהיה 1 כאשר יש רצף תקין שגדול מ- m (במקרה שלנו $m=7$ ו- $n=8$ אך נדגיש כי המערכת תתמוך בכל פרמטרים תקינים שנכניס). בדקנו כי מוצא המערכת ישאר 1 מרגע קיום התנאי עד אשר הספירה תתאפס או Detecion Code ישתנה.
להלן תוצאות הבדיקה בתצורת WAVE:



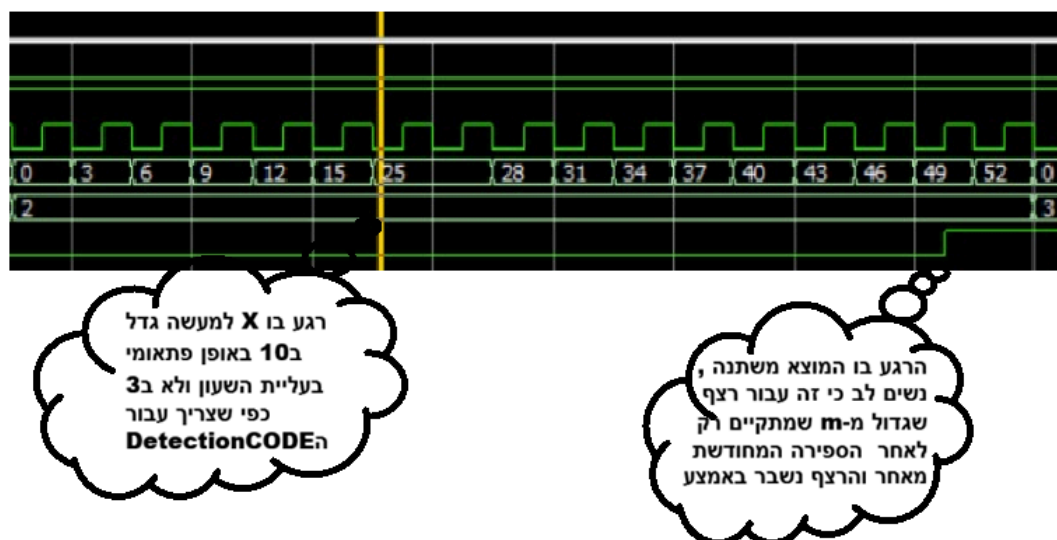
נשים לב כי תוצאות הסימולציה מדויקות. קיים עיכוב של שני מחזורי שעון מרגע הפעלת המערכת ועד תחילת הספירה הנובע מהצורך למלא את שרשרת הפליפ-פלופים על מנת לאפשר לרכיב Adder לחשב הפרש תקין. לכן רק במחזור ה-10 המוצא של המערכת יעלה ל-1. דבר זה מתאים במדויק לדרישות המערכת.

2. המקרה השני שבדקנו הוא מצב בו קו ה-Enable "נופל" וערכו משתנה ל-0 במהלך הספירה. במצב זה לפי Process מספר 3 יש השהייה ואין בדיקה עד אשר ה-Enable חוזר להיות 1.
בנוסף אנחנו לא ממשיכים לדגום ערכים חדשים אל תוך הפליפ-פלופים.
בדקנו מה קורה לאחר מכן כאשר ה-X ממשיך להשתנות בערכו ולגדול. למרות ש-Enable שווה ל-0 ואין זה ישפיע על המערכת כאשר הקו בקרה יעלה חזרה ל-1.
להלן תוצאות הבדיקה בתצורת WAVE:



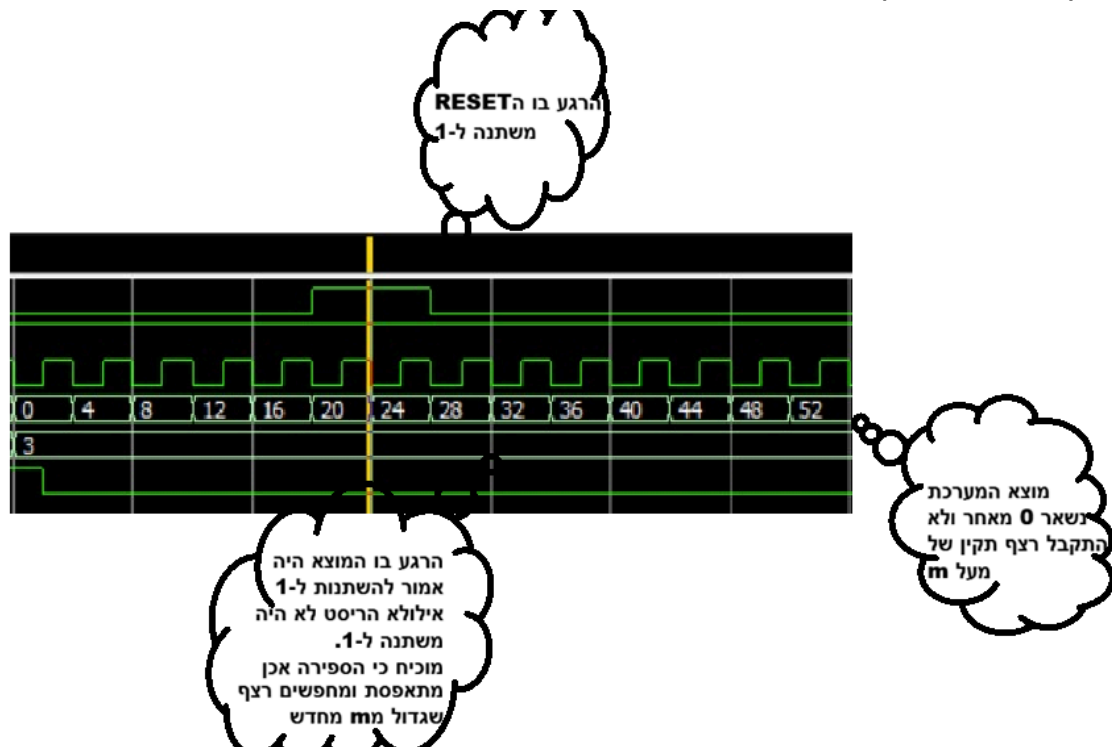
נשים לב כי תוצאות הסימולציה מדוייקות. ברגע ש-Enable יורד ל-0 ישנה עצירה בבדיקת קיום התנאי וגם בתהליך הדגימה לכן המערכת "קופאת" אבל ערכי X בtestbench ממשיכים להשתנות אך למעשה לא נדגמים והמערכת אינה פועלת עליהם אלא שומרת את הערכים הישנים. כתוצאה מכך, כאשר Enable עולה ל-1 מבוצעת בדיקה בין $X=12$ ל- $X=6$. מאחר וההפרש לא מקיים את התנאי עבור Detection Code ששווה ל-1 מתאפס המונה.

3. המקרה השלישי אותו בדקנו הוא מה קורה כאשר ה- X קופץ באופן פתאומי לערך אחר, התנאי נשבר לפעם אחת ולאחר מכן חוזר להיות תקין, איך זה משפיע על ספירת הרצף ומוצא המערכת. להלן תוצאות הבדיקה בתצורת WAVE:



נשים לב כי תוצאות הסימולציה מדוייקות, ברגע ש- X עולה לערך 25 ההפרש בין 2 הדגימות הוא 10 ולא 3 כפי ש-Detection Codew ציפה לראות. הספירה למעשה מתאפסת, ומתחילים לספור מחדש עד אשר הרצף יהיה גדול ממש m . ניתן לשים לב כי לקראת סוף הדוגמה זה אכן קורה ונשים לב כי לאחר 8 בדיקות חדשות של הפרשים המוצא עולה ל-'1' לוגי כצפוי.

4. המקרה הרביעי והאחרון הוא מצב בו קו הבקרה "איפוס" (Reset) מופעל, נבדוק איך זה משפיע על הספירה ומוצא המערכת.
להלן תוצאות הבדיקה בתצורת WAVE:



נשים לב כי תוצאות הסימולציה נכונות. אילו לא קו הבקרה reset היה משתנה, היינו מקבלים רצף תקין באורך הגדול מ-m ומוצא המערכת היה משתנה ל-1.
מאחר וReset עלה ל-1 הספירה התאפסה והספירה החלה מחדש.